

PROJECT-8

REQUIRED DATASET:

1. Charging stations data:
 - i. OpenChargeMap API (Local Data Bonus).
 - ii. Use a dense dataset of charging stations from Europe.
2. Vehicle Data:
 - i. Specs for a specific Car (e.g., Hyundai Ioniq 5: Range 480km, Battery 77.4kWh).

EXTERNAL RESOURCES:

- o Research the Shortest Path Problem with Resource Constraints (SPPRC).
- o Study battery consumption models (impact of speed and elevation)

CONSTRAINTS AND OBJECTIVE FUNCTION:

Constraints:

- o Hard: *Battery level* > 0% at all times.
- o Soft: Prefer "Fast Chargers" over "Slow Chargers." Minimize total trip duration (Driving Time + Charging Time).

Objective Function:

- o Minimize $\text{TotalTime} = \sum(\text{Drive_Time}) + \sum(\text{Charge_Time})$.

Cost Function:

$$\sum(\text{Drive_Time}) + \sum(\text{Charge_Time}) .$$

TASKS:

DATA + STATE DESIGN (4 DEVS):

1. Collect charging station data.

1 Project-8

2. Road Network (Transition Model for cities) eg:

```
road_network = {
  "Algiers":  [("Setif", 180)],
  "Setif":    [("Algiers", 180), ("Constantine", 130), ("Batna", 110)],
  "Batna":    [("Setif", 110), ("Constantine", 120)],
  "Constantine": [("Setif", 130), ("Batna", 120)]
}
```

3. Define Electric vehicle Specs.

4. Model (node,Battery %) state.

5. Implement Charge Action.

6. Node Expansion Logic.

UI (WEBSITE) (2 DEVS):

- Map Interface (Leflet.js)
- Route Overlay & chargin stations markers with estimated charge times.
- (Start city , end city)
- **Later on**: A "Battery Profile" graph (*Y-axis = Battery %*, *X-axis = Distance*) showing the charge draining and refilling along the route.

ALGORITHMS:

Breadth-First-Search (1 Dev):

Implement Bfs to serve as a baseline (this will likely fail or run out of memeory).

Greedy-First-Search (2 Devs):

Always drive towards the goal until battery is low, then drive to nearest charger. (Show how this can get stuck in a "dead end").

Heuristic $h(n) = \text{Ecludiean Dist} / \text{AvgSpeed}$.

Cost Function = $\sum(\text{Drive_Time}) + \sum(\text{Charge_Time})$.

Constrained A* Search (3 Devs):

Heuristic $h(n) = \text{Ecludiean Dist} / \text{AvgSpeed}$.

Cost Function = $\sum(\text{Drive_Time}) + \sum(\text{Charge_Time})$.

- Augment the state: State = (Node, BatteryLevel).
- Logic: If $\text{Battery} < \text{DistToNextNode}$, that edge is infinite cost (blocked).